walrus
memory

Continuity Keeper

The story bible your AI can't contradict — portable, self-enforcing, on Walrus.

`Walrus Mainnet` `19 blobs` `account 0x8dd9…c9d7`

THE PROBLEM

AI co-writers contradict their own canon.

CHARACTER

A character killed in Chapter 7 speaks again in Chapter 12.

WORLD

A stated magic rule quietly breaks itself two scenes later.

PLOT

An object destroyed on the page reappears intact later.

TIMELINE

Events resolve in an order the story has made impossible.

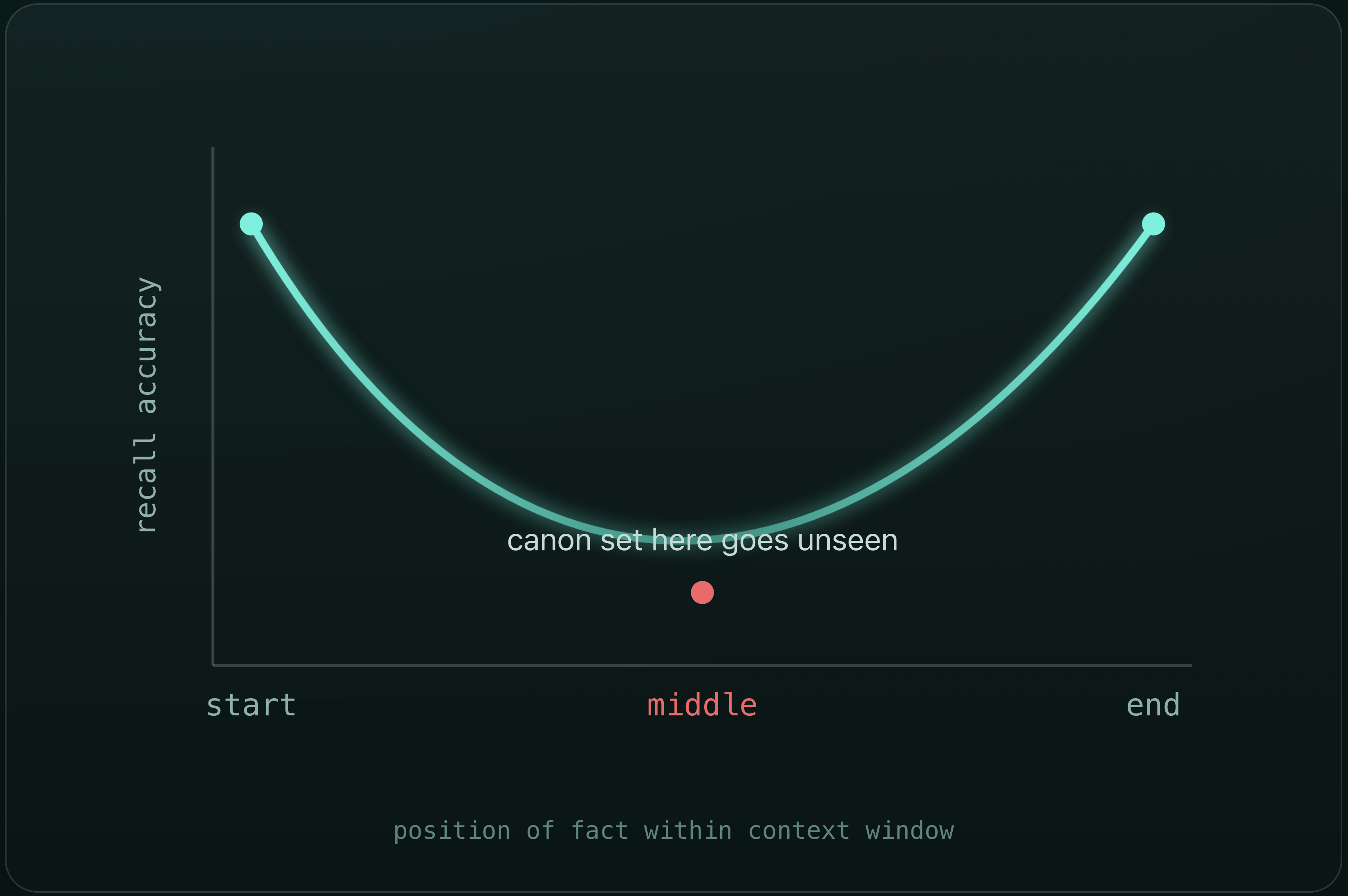
Happens every scene, every session — once a story gets long enough to lose track of itself.

WHY IT HAPPENS

Not a prompting bug — a known failure of attention.

"Lost in the Middle" (Liu et al., TACL 2024): LLM accuracy is highest when facts sit at the start or end of context, and drops >30% when they sit in the middle.

The U-shaped curve replicates across **6 model families**. Canon set early stops being "seen" around chapter 10–15 — and the model contradicts it, confidently.



IT'S A REAL, PAID PAIN

“The central unsolved problem of AI-assisted long-form fiction.”

— Novarrium

Sudowrite

Story Bible

Structured canon store bolted onto the editor.

NovelCrafter

Codex

A living archive of evolving character & world states.

Novarrium

Logic-Locking

Explicit constraints checked against new prose.

A whole comparison market exists — "Best AI for Novel Continuity Checking 2026: 5 tools compared."

WHY TODAY'S FIXES FALL SHORT

Both approaches leave the writer holding the burden.

A · MANUAL TRACKING

Pasting character sheets into every prompt.

The upkeep grows with the book. Every new scene is one more thing you must remember to re-paste — and one more chance to forget.

B · VENDOR STORY BIBLES

Locked inside one app, enforced by hand.

The vendor holds your unpublished manuscript. Open a different tool and your canon doesn't come with you.

THE SOLUTION

One system prompt + a tiny CLI.

1

Recall

Pull the relevant canon before writing the next scene.

→

2

Contradiction-guard

Stop and flag conflicts — never silently override.

→

3

Supersede

Retire outdated facts for real, so recall stays current.

Grounded in the agent-memory literature — A-MEM (structured notes + memory evolution) and CoALA (a memory taxonomy for language agents).

WHY WALRUS · THE MOAT

Only Walrus delivers all four at once.

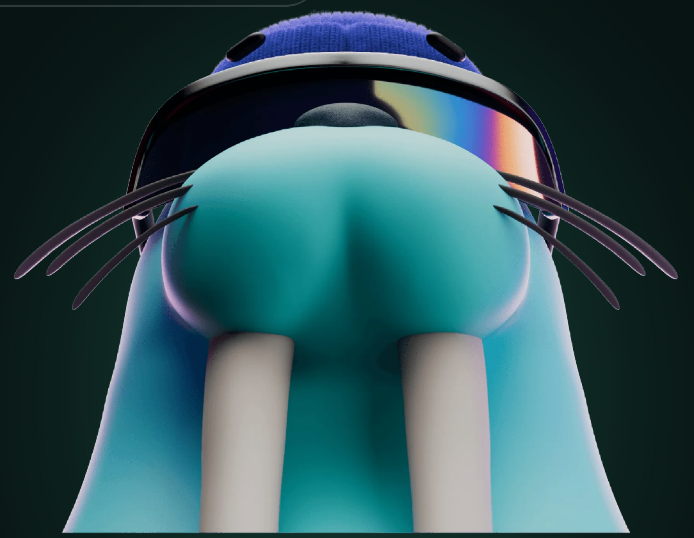
Portable
Recall from any MCP client — Claude Code, Cursor, Codex, Gemini CLI.

Self-enforcing
Recall and guard run automatically — not on the writer's discipline.

Owned & private
You hold the keys. Blobs are SEAL-encryptable — the manuscript stays yours.

Provable history + current truth
Superseded blobs persist on Walrus; recall returns only what's current.

This bundle is **impossible** on a plain vector DB or a text file.



HOW SUPERSESSION WORKS

Forget the namespace, keep the blob.

NAMESPACE

One entity, one tiny namespace of facts.

`elara/*`

→

FORGET

On a change, drop the namespace from recall.

`forget elara/*`

→

REWRITE

Write the new current facts into the namespace.

`elara/status = dead`

The old "alive" fact stops surfacing in recall — but its blob stays on Walrus as immutable history. Current truth for the agent, full history on-chain.

PROOF ON WALRUS MAINNET

19 blobs

2 supersessions

1 production relayer

AGENT ACCOUNT

0x8dd9d47183f88a1ab70515bed7494685487458614131e2004f7d18f1d3b9c9d7

Story used: an original short piece, "Saltglass."

BEFORE SUPERSEDE

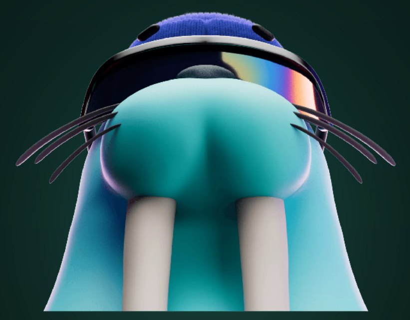
`recall("Elara") → ALIVE (Ch.1)`

↓ supersede

AFTER SUPERSEDE

`recall("Elara") → only DEAD (Ch.7)`

The retired "alive" blob still resolves on Walruscan.



Portable. Self-enforcing. Yours.

One prompt, any tool, on Walrus.

walrus
memory

`github.com/yukitran03/continuity-keeper`

Judges: clone it, point any MCP client at your Walrus account, and watch recall stay consistent.